# Pizza Sales Report

## ANNUAL PERFORMANCE ANALYSIS

Insights & Strategic Growth

Presented by Rakesh Singh

# Data Architecture & Schema

This report analyzes pizza sales data to uncover key insights on revenue, customer behavior, and product performance, enabling data-driven business decisions.

## order_details

Columns:
order_details_id, order_id, pizza_id, quantity

## orders

Columns:
order_id, date, time

## pizzas

Columns:
pizza_id, pizza_type_id, size, price

## types

Columns:
pizza_type_id, name, category, ingredients

# Case Study Questions

QUESTION 01

What were the specific orders placed on 2015-01-01?

QUESTION 02

Which distinct pizza categories are currently available?

QUESTION 03

What is the total number of pizzas ordered in this period?

QUESTION 04

What is the average transaction value per customer?

QUESTION 05

How does each pizza type contribute to total revenue?

QUESTION 06

Category generated the maximum revenue.

Question 07

Identifying the pizza that has been ordered the least number of times.

# 1. Retrieve all orders placed on 2015-01-01.

```
SELECT order_id FROM orders
WHERE date = '2015-01-01';
```

# 2. List all distinct pizza categories available.

```
SELECT DISTINCT category
FROM types;
```

# 3. Find total number of pizzas ordered.

```
SELECT
     SUM(quantity) AS Total_pizza_ordered
FROM order_details;
```

# 4. Get the top 5 most frequently ordered pizza types.

```
SELECT
     TOP 5 t.name,
     SUM(od.quantity) AS Freq_ordered_pizza
FROM order_details od
     JOIN pizzas p ON
     od.pizza_id = p.pizza_id
     JOIN types t ON
     p.pizza_type_id = t.pizza_type_id
GROUP BY t.name
ORDER BY Freq_ordered_pizza DESC;
```

# 5. Show order_id and total pizzas ordered in each order.

```sql
SELECT
  order_id,
  SUM(quantity) AS Pizza_Ordered
FROM order_details
GROUP BY order_id
ORDER BY Pizza_Ordered DESC;
```

# 06. Hourly Order Distribution Analysis

Identifying peak ordering times throughout the day.

```sql
SELECT
  DATEPART(HOUR, time) AS Time,
  COUNT(order_id) AS Orders_placed
FROM orders
GROUP BY DATEPART(HOUR, time)
ORDER BY Time;
```

# 07. Peak Ordering Day Analysis

Identifying the day of the week with the maximum volume of orders.

```sql
SELECT TOP 1 DATENAME(WEEKDAY, date) AS Day,
  COUNT(order_id) AS Orders_placed
FROM orders
GROUP BY DATENAME(WEEKDAY, date)
ORDER BY Orders_placed DESC;
```

# 08. Daily Order Time Extremes

Retrieving the earliest and latest order timestamps for every recorded date.

```sql
SELECT date,
CAST(MIN(time) AS TIME(0)) AS Earliest_Ord,
CAST(MAX(time) AS TIME(0)) AS Latest_Ord
FROM orders
GROUP BY date
ORDER BY date ASC;
```

# 09. Revenue by Pizza Type

Find the total revenue generated from each pizza type.

```sql
SELECT
  t.name,
  ROUND(SUM(quantity*price),2) AS Total_Revenue
FROM order_details od
  JOIN pizzas p ON od.pizza_id = p.pizza_id
  JOIN types t ON p.pizza_type_id = t.pizza_type_id
GROUP BY t.name;
```

# 10. Which pizza category generated the maximum revenue?

Identifying the top-performing pizza category based on total sales value.

```sql
SELECT
  TOP 1 t.category,
  ROUND(SUM(od.quantity * p.price), 2) AS Total_Revenue
FROM order_details od
  JOIN pizzas p ON od.pizza_id = p.pizza_id
  JOIN types t ON p.pizza_type_id = t.pizza_type_id
GROUP BY t.category
ORDER BY Total_Revenue DESC;
```

# 11. Average Pizzas per Order

Calculating the mean volume of pizzas across all transactions in the dataset.

```sql
WITH total_orders AS(
  SELECT
    order_id,
    SUM(quantity) AS total_orders
  FROM order_details
  GROUP BY order_id
)
SELECT
  CAST(AVG(total_orders) AS FLOAT) AS Avg_Orders
FROM total_orders;
```

# 12. Top 3 Highest Spending Orders

Identifying the individual transactions that resulted in the highest total revenue.

```sql
SELECT
  TOP 3 od.order_id,
  ROUND(SUM(od.quantity * p.price), 2) AS Spendings
FROM order_details od
  JOIN pizzas p ON od.pizza_id = p.pizza_id
GROUP BY od.order_id
ORDER BY Spendings DESC;
```

# 13. Cumulative Daily Revenue

Calculating the running total of revenue generated throughout the day.

```sql
SELECT
  o.date,
  CAST(o.time AS TIME(0)) AS Time,
  ROUND(SUM(od.quantity * p.price), 2) AS Total_Rev,
  ROUND(SUM(SUM(od.quantity * p.price)) OVER(PARTITION BY o.date ORDER BY
  o.time), 2) AS Cumulative_Rev
FROM order_details od
  JOIN orders o ON od.order_id = o.order_id
  JOIN pizzas p ON od.pizza_id = p.pizza_id
GROUP BY o.date, o.time;
```

# 14. Least Popular Pizza Identification

Identify the specific pizza variety that has been ordered the least number of times across the dataset.

```sql
SELECT
  TOP 1 t.name,
  COUNT(od.order_id) AS Orders
FROM order_details od
  JOIN pizzas p ON od.pizza_id = p.pizza_id
  JOIN types t ON p.pizza_type_id = t.pizza_type_id
GROUP BY t.name
ORDER BY Orders;
```

# 15. Pizza Category Revenue Share

Analyzing the percentage contribution of each pizza category to the overall revenue.

```sql
WITH category_revenue AS(
  SELECT
    t.category,
    ROUND(SUM(od.quantity * p.price), 2) AS Cat_Rev
  FROM order_details od
    JOIN pizzas p ON od.pizza_id = p.pizza_id
    JOIN types t ON p.pizza_type_id = t.pizza_type_id
  GROUP BY t.category
)
SELECT
  category, Cat_Rev,
  ROUND(SUM(Cat_Rev) OVER(), 2) AS Total,
  ROUND(Cat_Rev * 100.0 / SUM(Cat_Rev) OVER(), 2) AS Pct_Contri
FROM category_revenue;
```

# The End.

# Thank You

Pizza Sales Analysis Report

-- Rakesh Singh